KakeyaLattice: Nested-Lattice KV-Cache Compression with Kakeya-Style Discrete Codebooks

Allen Li
Individual researcher
Email: AllenL329@gmail.com

April 24, 2026

Abstract

We introduce KakeyaLattice, a KV-cache compression codec for transformer LLMs that constructs a *discrete Kakeya cover* over the direction sphere via a Zamir–Feder nested-lattice quantiser [24]. The paper covers two concrete instantiations of a single codec family: a D_4 nested lattice variant and an E_8 nested lattice variant, sharing the same nine-step pipeline (unit-norm factorisation, Sylvester–Hadamard rotation, per-vector adaptive $q_{\max}$, joint scale, lattice closest-point, clamp). The key design innovation is adaptation to the measured non-Gaussian structure of real LLM KV activations (sub-Gaussian body, per-coordinate heavy tail after rotation, coordinate anisotropy up to 4.71× on Qwen3-4B post-QK-norm K); without these levers the predicted shaping gain does not manifest. The lattice Voronoi cells replace the cube cells of $\mathbb{Z}^N$, trading $G(\mathbb{Z}^N) = 1/12$ for $G(D_4) \approx 0.0766$ [26] or $G(E_8) \approx 0.0717$.

Measured results are live-vLLM on NVIDIA H200 under two protocols. Under *snapshot* evaluation the D_4 variant wins 12/12 on K-MSE (10–36% better) and 9/12 on $|\Delta\text{ppl}|$ across four open-source models at three near-matched bit tiers; the theoretical $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ shaping ratio is recovered to within $\sim 1\%$ in three independent environments. Under *in-forward rigorous* evaluation ($n = 32$, 95% CI, no-boundary) the E_8 variant reduces $|\Delta\text{ppl}|$ by 28–53% across three deployable models at $Q \in \{4, 10\}$, with +1.3 to +2.0 dB per-layer K-MSE gain over D_4 — 4–6× the +0.29 dB theoretical minimum, because E_8's two-coset structure handles coarse-quantisation outliers better than D_4's single parity flip. Cost: +16–32 packed bits/vector (−5 to −7% CR at iso-q_{range}) and 1.56–1.80× encode latency. Long-context retrieval (Needle-in-a-Haystack at 16k) is preserved on Qwen3-4B and Gemma-4-E4B. Strict-GPU, no mock / simplification / fallback / overfit; bit-level regression gated by a pinned `sha256` frozen-parity test (8 hashes). Code, per-passage JSON, four per-architecture attention hooks, and the multi-model / NIAH / latency harnesses are released under Apache-2.0.

1 Introduction

The key–value (KV) cache is the dominant memory constraint in production LLM inference [42, 43, 47]. At context length T with L layers, H_{kv} heads, head dim d, the bf16 KV footprint is $4LH_{kv}dT$ bytes — 18 GiB per user for Qwen3-4B at $T = 128{,}000$, several hundred GiB each for the April-2026 flagship open-source models (DeepSeek-V3.1, Kimi-K2, Qwen3-235B, GLM-4.6). Compressing this object post-hoc (without re-training) is one of the highest-leverage algorithmic targets in current serving infrastructure.

Lattice shaping, unexploited. Every production KV codec we know of quantises the Hadamard-rotated residual *per coordinate*, implicitly using the cube cell of $\mathbb{Z}^N$ whose normalised second moment is $G(\mathbb{Z}^N) = 1/12$ [24, 26]. Shannon's bound for lattice Λ is

$$\text{MSE}_\Lambda = G(\Lambda) \cdot 2^{-2b}, \quad G(\Lambda) \le G(\mathbb{Z}^N), \tag{1}$$

so using $\Lambda \in \{A_2, D_4, E_8, \Lambda_{24}\}$ with $G(\Lambda) \in \{0.0802, 0.0766, 0.0717, 0.0658\}$ yields 0.17/0.37/0.65/1.03 dB of unconditional *shaping gain* over $\mathbb{Z}^N$ at matched rate. This is free MSE reduction, constrained only by lattice encode cost.

The codec as a discrete Kakeya cover. The classical Kakeya problem [1, 2, 7, 10, 11, 21] asks for the minimum measure of a set in $\mathbb{R}^D$ containing a unit segment in every direction. A KV-cache codec reconstructing each x_i to ℓ_2-error ε must cover a tube of radius ε around each direction in $\Theta = \{x_i / \|x_i\|\} \subset S^{D-1}$ — exactly what the Kakeya maximal-function conjecture bounds. KakeyaLattice's codebook

realises the discrete version of this cover: the tensor-product $D_4^{\otimes D/4}$ (or $E_8^{\otimes D/8}$) Voronoi cells, scaled adaptively per vector by $q_{\max}/q_{\text{range}}$, tile $\mathbb{R}^D$ so that every direction falls within one cell. The cover's second-moment optimum is unconditional at the codec's block dimension, replacing the conditional Kakeya lower-bound used in our prior low-rank codec [33].

1.1 Contributions

(i) A discrete Kakeya cover with working $O(D)$ GPU decoder (Conway–Sloane closest-point + one parity flip [25]). (ii) Non-Gaussian adaptation as the main engineering lever: five pre-processing levers (§3.2) against the specific mismatches measured in Qwen3-4B K ($|\text{excess kurt}| = 0.84$, RMS Wasserstein-$2/\sigma = 0.65$ per dim, variance ratio 4.71×), all past their Gaussian-check gates. (iii) An unconditional rate–distortion boundary at the codec's block dimension via D_4 / E_8 second-moment optimality. (iv) Architecture-independent gain across four heterogeneous transformer families (Qwen3-4B, DeepSeek-R1-Distill-1.5B, Gemma-4-E4B at head_dim = 256, GLM-4-9B-Chat): K-MSE ratio within $\sim 1\%$ of 0.919 on every model. (v) Strict-GPU live-vLLM evaluation under two protocols chosen per question: snapshot (§4; $n = 4$) for the D_4 headline in §5; in-forward rigorous (§4; $n = 32$, 95% CI, no-boundary) for the E_8-vs-D_4 comparison in §6; every results-table caption cites its protocol. (vi) 128k KV storage reduction of 2.5–2.8× balanced / 1.66–1.73× near-lossless with both K and V compressed ($|\Delta\text{ppl}| < 1\%$, top-1 $\geq 98\%$ at the near-lossless point).

Paper structure. §2 places the codec inside the Kakeya–Brascamp–Lieb–Tropp chain and states the Voronoi proposition; §3 gives the nine-step algorithm and the five-lever design philosophy; §4 the two measurement protocols; §5 the snapshot-protocol D_4 headline; §6 the in-forward rigorous E_8 results; §7 caveats; §8 framing as a near-matched-rate drop-in PR over TurboQuant; §9 related work. The Kakeya framing is literature-traceable, not a new theorem.

2 Design philosophy: the Kakeya–Brascamp–Lieb–Tropp chain

Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^D$ be a block of n cached K vectors at head dimension D (post-QK-norm pre-RoPE), inducing a direction set $\Theta = \{x_i / \|x_i\|\} \subset S^{D-1}$. We take squared-$\ell_2$ distortion $\rho(x, \hat{x}) = \|x - \hat{x}\|_2^2$ (see §10 on the MSE vs. inner-product choice). A codec reconstructing every x_i to ℓ_2-error ε must cover a tube of radius ε around every direction in Θ; the minimum cost of such a tube-union is exactly what the classical Kakeya maximal-function conjecture bounds [1, 2, 7, 10]. KakeyaLattice realises the discrete version of this cover explicitly via the D_4 (or E_8) Voronoi tessellation.

Proposition 2.1 (Kakeya-conditional rate–distortion lower bound, informal)**.** *Let $\Theta \subset S^{D-1}$ have metric-entropy dimension $d_\delta(\Theta)$. Conditional on the Kakeya maximal-function conjecture in dimension D, the minimum byte budget for ℓ_2-error $\varepsilon \|x_i\|$ satisfies $B(\varepsilon, n, D) = \Omega(n \log_2 \frac{1}{\varepsilon} + d_\delta(\Theta) \log_2 \frac{1}{\varepsilon})$. At $D = 3$ the bound is unconditional [21]; at $D \geq 4$ it remains conditional.*

The bound sits at the deep end of a published four-link chain: *multilinear Kakeya endpoint* [14] → *Brascamp–Lieb equivalence* [16] → *matrix concentration* (Tropp's matrix Chernoff via Lieb concavity [22, 17]) → *lattice second-moment optimality* [24, 26]. The codec uses the chain's shallow end (link 4), which is *unconditional* at the codec's block dimension:

Proposition 2.2 (D_4 Voronoi cells as a restricted Kakeya cover)**.** *After unit-norm factorisation and Sylvester–Hadamard rotation, let $\Lambda = D_4^{\otimes K}$ with $K = D/4$ scaled by $q_{\max}/q_{\text{range}}$. Its Voronoi cells direction-cover S^{D-1} with covering radius $\varrho(\Lambda)^2 \leq q_{\max}^2 \cdot G(D_4)/[q_{\text{range}}^2 \cdot (2q_{\text{range}} + 1)^{4K/D}]$, where $G(D_4) \approx 0.0766 < G(\mathbb{Z}^4) = 1/12 \approx 0.0833$. The ratio $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ is the MSE shaping gain (+0.367 dB). The same statement holds for $\Lambda = E_8^{\otimes K}$ at $K = D/8$ with $G(E_8) \approx 0.0717$ (+0.65 dB over $\mathbb{Z}^8$, or +0.29 dB over D_4).* □

The proof is immediate from the second-moment formula for Voronoi cells [26] and the Gaussian channel-coding view [24, 27, 29]; it drives the central empirical prediction (0.919 K-MSE ratio) tested in §5. Our use of Kakeya is literature-traceable *framing*, not a new harmonic-analysis theorem: the D_4-lattice bound is pure lattice geometry, the direction set Θ is a finite discrete instance (not the continuous Kakeya hard case), and the MSE bound is multiplicative rather than Hausdorff-dimensional.

3 The KakeyaLattice codec

3.1 Nine-step pipeline, parameterised by lattice choice

The D_4 nested lattice variant and the E_8 nested lattice variant share the same nine-step pipeline. Only the choice of lattice Λ and its block dimension $\dim \Lambda$ differ.

Algorithm 1 KakeyaLattice encode/decode (single vector), parametric in $\Lambda \in \{D_4, E_8\}$

Input: $x \in \mathbb{R}^D$, $q_{\text{range}} \in \mathbb{Z}_{\geq 1}$, lattice Λ with $\dim \Lambda \in \{4, 8\}$, $D \equiv 0 \pmod{\dim \Lambda}$.

encode

$r \leftarrow \|x\|_2; \quad u \leftarrow x/r$ ▷ unit-norm factorisation
$y \leftarrow Hu/\sqrt{D}$ ▷ Sylvester–Hadamard rotation
$q_{\max} \leftarrow \max_i |y_i|$ ▷ per-vector adaptive scale
$\tilde{y} \leftarrow y \cdot q_{\text{range}}/q_{\max}$ ▷ scale onto lattice grid
for each block $b = 1, \ldots, D/\dim \Lambda$ of $\dim \Lambda$ coords **do**
  if $\Lambda = D_4$ **then**
    $z_b \leftarrow \text{ClosestPoint}_{D_4}(\tilde{y}_b)$ ▷ Conway–Sloane Alg. 4 [25]
  else if $\Lambda = E_8$ **then**
    $z_b \leftarrow \text{ClosestPoint}_{E_8}(\tilde{y}_b)$ ▷ Conway–Sloane Alg. 5 [25]
  end if
  $z_b \leftarrow \text{clamp}(z_b, -q_{\text{range}}, +q_{\text{range}})$
end for
Store: lattice indices $\{z_b\}$ + two fp16 scalars $(r, q_{\max})$.

decode

$\hat{y} \leftarrow \{z_b\} \cdot q_{\max}/q_{\text{range}}$ ▷ reconstruct rotated residual
$\hat{u} \leftarrow H\hat{y}/\sqrt{D}$ ▷ Hadamard is self-inverse at this scale
$\hat{x} \leftarrow \hat{u} \cdot r$ ▷ undo unit-norm factorisation
Return: $\hat{x}$.

D_4 closest-point. $D_4 = \{z \in \mathbb{Z}^4 : \sum_i z_i \equiv 0 \pmod 2\}$. Given $\tilde{y} \in \mathbb{R}^4$, round to $f = \text{round}(\tilde{y})$. If $\sum_i f_i$ is even, $f \in D_4$. Otherwise flip the coordinate with largest $|\tilde{y}_i - f_i|$ by ± 1 in the sign of $\tilde{y}_i - f_i$. Cost: $O(1)$ per block; warp-uniform except for a single parity-flip branch.

E_8 closest-point. $E_8 = D_8 \cup (D_8 + \frac{1}{2}\mathbf{1})$ where $D_8 = \{z \in \mathbb{Z}^8 : \sum_i z_i \equiv 0 \pmod 2\}$. Compute two candidates: the closest D_8 point to $\tilde{y}$, and the closest D_8 point to $\tilde{y} - \frac{1}{2}\mathbf{1}$ translated back by $+\frac{1}{2}\mathbf{1}$; return whichever is closer to $\tilde{y}$ in squared distance. Cost: two D_8 evaluations plus one min, $\approx 2.5\times$ per-block work vs D_4 but the block count halves, giving a net ≈ 1.56–$1.80\times$ encode cost on an NVIDIA H200 (Section 6.2).

Bit budget (exact vs. slot-packed). At parameter q_{range} the D_4 lattice contained in $[-q_{\text{range}}, +q_{\text{range}}]^4$ has $(2q_{\text{range}} + 1)^4/2$ points (parity constraint halves the $\mathbb{Z}^4$ count), with $O(q_{\text{range}}^3)$ boundary correction absorbed into the clamp step. The *exact* (information-theoretic) per-block rate is

$$\text{bits per } D_4 \text{ block} \;=\; 4\log_2(2q_{\text{range}} + 1) - 1, \tag{2}$$

multiplied by $K = D/4$ blocks plus two fp16 overhead scalars per vector. Deployment slot formats address integer bits per block, so the *packed* per-block cost is $\lceil 4\log_2(2q_{\text{range}} + 1) - 1 \rceil$ bits — typically one bit above the exact value. At $D = 128$ the D_4 variant at $q_{\text{range}} = 152$ therefore uses 1056.3 bits exact but $33 \cdot 32 + 32 = 1088$ bits slot-packed; every bit column in this paper's tables is the *packed* value, which is what a production PR would actually consume. The same per-q_{range} table (exact, packed, per-head bytes) is in Appendix C.

For the E_8 nested lattice variant: the two cosets (integer and half-integer) recover the full $\mathbb{Z}^8$ density within a parity constraint, so $E_8 \cap [-q_{\text{range}}, +q_{\text{range}}]^8$ has $\approx (2q_{\text{range}} + 1)^8$ points at large q_{range}:

$$\text{bits per } E_8 \text{ block} \;=\; 8\log_2(2q_{\text{range}} + 1), \tag{3}$$

multiplied by $K = D/8$ blocks plus the same two fp16 scalars. Packed (integer-aligned) per-vector cost is $16\lceil 8\log_2(2q_{\text{range}} + 1)\rceil + 32$. The packed $E_8 - D_4$ gap is therefore $+32$ bits per vector at low q_{range} (parity saving visible after rounding) and $+16$ bits at high q_{range} (parity saving absorbed into integer alignment); the per-q_{range} gap table is in Table 7. The exact (unpacked) $D_4 \to E_8$ overhead is the constant $+D/4$ bits per vector for the lost parity saving, independent of q_{range}; the packed gap differs from this constant purely because ceiling arithmetic collapses fractional bits into whole bits.

3.2 The design role of each step

The nine steps factor into *five engineering levers plus one lattice*. The five levers are not there for structural elegance; each one closes a specific measured gap between real KV activations and the i.i.d. isotropic source model that Shannon's rate–distortion bound (1) assumes. The non-Gaussian audit of Qwen3-4B post-QK-norm K summarised in §1 identifies exactly four such gaps (per-vector magnitude spread, coordinate-wise heavy tail after rotation, per-vector envelope variability, coordinate-wise anisotropy); against each, one of the five levers is the mechanism that closes it. The sixth ingredient is the *structural lever*: the lattice. The D_4 nested lattice variant gives the fixed $+0.367$ dB shaping gain on top of an already-normalised input. The E_8 nested lattice variant substitutes E_8, which is the $\mathbb{R}^8$ second-moment optimum (as D_4 is for $\mathbb{R}^4$) and brings an additional $+0.29$ dB of theoretical shaping gain. The lattice upgrade is strictly modular: none of the five engineering levers changes, so the E_8 variant inherits every non-Gaussian-source correction the D_4 variant makes, and the E_8 benefit stacks on top.

The five engineering levers (each closing one measured non-Gaussian gap) are:

- **Unit-norm factorisation** (steps 2, 12) absorbs per-vector magnitude spread (largest $> 10\times$ median on every tested model), reducing the grid to the bounded direction $u = x/\|x\|$ at cost of one fp16 scalar per vector.
- **Sylvester–Hadamard rotation** (steps 3, 11) whitens coordinate-wise anisotropy: the raw direction still has variance ratios up to $4.71\times$, which after Hadamard drops below $2\times$ (Qwen3-4B K). The rotation is arithmetic-exact in bf16 [31, 30] and self-inverse.
- **Per-vector adaptive** $q_{\max}$ (step 4) absorbs heavy per-coordinate tails; swapping per-vector for global scale costs $\sim 4\times$ K-MSE.
- **Joint quantisation** (step 6): all $D/\dim\Lambda$ blocks of one vector share a single $q_{\max}$, so bit-rate allocation tracks the per-vector envelope rather than block-local maxima (within-vector block variance ratios reach $3\times$).
- **Matched bit budget** (step 7 clamp plus (2)): clamp absorbs the $O(q_{\text{range}}^3)$ boundary correction. Choosing q_{range} to track production slot sizes (e.g. $q_{\text{range}} = 152$: exact 1056.3 bits per $D = 128$ vector, slot-packed to 1088) keeps the codec slot-aligned; the ceiling overhead is the 3–11% premium discussed in §8.

The sixth ingredient is the structural lever: the lattice itself ($+0.367$ dB from D_4, $+0.65$ dB from E_8 over $\mathbb{Z}^N$, per Prop. 2.2). The five engineering levers are already at their theoretical optima and cannot be improved individually; the lattice is the only knob left, which is why $D_4 \to E_8$ is our upgrade path (§6).

3.3 Implementation and compliance

Canonical Python classes `V14KakeyaZamirLatticeGPU` and `V15KakeyaZamirE8GPU` are released under Apache-2.0 in the `kakeyalattice` package [32]. The codec is strict-GPU (element-wise PyTorch CUDA, no CPU round-trip). The repository enforces a strict ban list at test time: **no mock** (real vLLM, HF weights, WikiText-103, FlashAttention bf16), **no simplification** (code path identical across every model), **no fallback** (non-divisible head dim raises `ValueError`; silent-hook passthrough aborts via fire-count guard), **no overfit** (boundary $k = 2$ and canonical $q_{\text{range}} \in \{10, 38, 152\}$ are constants across every model).

4 Experimental methodology

Models and environment. Four open-source decoder-only transformer models, chosen for architectural heterogeneity (Table 1). All runs on NVIDIA H200 (80 GiB, CUDA 13.0) via vast.ai with vLLM `0.19.2rc1.dev100+gf946659ff` [48] and transformers 5.5.2; bf16 weights, FlashAttention.

Table 1: Benchmark corpus.

Model	Params	L	n_{kv}·hd	Attention family
Qwen/Qwen3-4B	4 B	36	$8 \cdot 128$	GQA + per-head QK-norm
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	1.5 B	28	$2 \cdot 128$	GQA + halfsplit RoPE
google/gemma-4-E4B	4 B	24	$2 \cdot 256$	Hybrid sliding + full, kv-share
zai-org/GLM-4-9B-Chat	9 B	40	$2 \cdot 128$	GQA + adjacent-pairs RoPE + QK-norm

Snapshot-mode capture and replace. Two-pass protocol. *Pass 1*: real vLLM prefill at ctx $= 2048$; a per-architecture attention-module monkey-patch (`snapshot_hook.py`) captures post-QK-norm pre-RoPE

K, V in strict-GPU fp32. *Offline*: every non-boundary layer's K (and V, when enabled) is codec-round-tripped on GPU. *Pass 2*: a second vLLM forward in which the same hook replaces freshly-projected K, V with the decoded tensors; prompt-logprobs over the $n_{\text{eval}} = 64$ window $[2048, 2112)$. A fire-count guard aborts on silent passthrough. This mirrors HuggingFace's `DynamicCache` two-pass pattern [49]: cache entries depend only on the codec applied to the clean Pass-1 capture, isolating codec *intrinsic* error from cross-layer accumulation. Boundary layers $\{0, 1, L-2, L-1\}$ are kept bf16 by default ($L_{\text{bdry}}/L \in \{10, 14, 17, 10\}\%$).

In-forward rigorous evaluation. Codec is applied *inside* every attention step before the KV-cache write; per-layer errors compound. Default $k = 0$ (no boundary protection); $k = 2$ only for codecs structurally undeployable at $k = 0$ (D_4 Q=2, TQ $b = 2$). Sample size $n = 32$ WikiText-103 passages per (model, codec, q) tuple, mean $\pm 95\%$ CI via Student-t. Harness: `benchmarks/rigorous_eval.py`; bit-level regression protected by a pinned `sha256` frozen-parity test (8 hashes). Snapshot vs. in-forward answer different questions (intrinsic error vs. deployment-realistic error); every results-table caption cites the protocol it was produced under.

Quality metrics. $\boldsymbol{K}$ **rel-MSE** $= \left\|K - \hat{K}\right\|_F^2 / \left\|K - \bar{K}\right\|_F^2$ (sample-size-independent, per-vector closed form); $\mathbf{|\Delta ppl|} = |\text{ppl}_{\text{alt}} - \text{ppl}_{\text{ref}}|/\text{ppl}_{\text{ref}}$ averaged over passages $\times n_{\text{eval}} = 64$; **top-1 pair agreement** = fraction of eval positions where alt / ref argmax coincide. We use $|\Delta\text{ppl}|$ rather than signed Δppl because Hadamard-rotated structured quantisation exhibits a de-biasing phenomenon [30] on well-regularised models, so the signed value crosses zero and is not monotone in codec quality.

5 Main results: D_4 variant, snapshot protocol

All tables in this section use the snapshot protocol defined in §4 ($n = 4$ passages, boundary-layer protection $k = 2$). The protocol isolates codec intrinsic error and is the correct instrument for the four claims tested here; in-forward, cross-layer-accumulated numbers (including E_8-vs-D_4 side-by-side) are the subject of §6.

We test four claims in sequence: (i) that the D_4 shaping gain translates to a monotone Pareto improvement across bit budgets at near-matched rate (§5.1); (ii) that the non-Gaussian adaptation is architecture-independent across four transformer families (§5.2); (iii) that the measured K-MSE ratio reproduces the theoretical 0.919 to within $\sim$1% (§5.3); and (iv) that the codec delivers usable KV-cache reduction at 128k context with both K and V compressed (§5.4).

5.1 Compression-rate Pareto on Qwen3-4B

Table 2 gives the full Pareto sweep at seven bit-tier pairings between TurboQuant $b \in \{2, \ldots, 8\}$ and KakeyaLattice $q_{\text{range}} \in \{2, 5, 10, 19, 38, 76, 152\}$ on four WikiText-103 passages at ctx $= 2048$, $n_{\text{eval}} = 64$, strict-GPU, under the snapshot protocol (§4). The pairings are *near-matched rate*, not strictly iso-rate: the KakeyaLattice row in each pair sits 3–11% above its TQ row (Section 8).

Table 2: Full Pareto: KakeyaLattice (D_4 variant) vs. TurboQuant $k8v4$ on Qwen3-4B, H200, vLLM 0.19.2. Protocol: *snapshot-mode*, $n = 4$ passages, boundary $k = 2$ (§4); see §6 for in-forward rigorous numbers. Bold rows are the three canonical operating points. CR = compression ratio vs. bf16. Bit column reports the *packed* (integer-aligned) per-vector cost; see Eq. (2) and Table 4 for the exact-vs-packed discussion.

bits	CR	Config	Δppl	|Δppl|	top-1	K rel-MSE
288	7.11×	TQ $b = 2$	+150.5%	150.5%	66.4%	5.6×10^{-1}
320	**6.40×**	D_4 **Q=2**	+30.3%	30.3%	**83.6%**	$\mathbf{1.9 \times 10^{-1}}$
416	4.92×	TQ $b = 3$	+2.22%	5.91%	90.6%	6.7×10^{-2}
448	4.57×	D_4 Q=5	+2.83%	2.83%	94.5%	3.1×10^{-2}
544	3.76×	TQ $b = 4$	+3.51%	4.22%	97.7%	1.2×10^{-2}
576	**3.56×**	D_4 **Q=10**	+1.01%	1.86%	97.3%	$\mathbf{7.8 \times 10^{-3}}$
672	3.05×	TQ $b = 5$	+1.24%	1.24%	98.4%	2.7×10^{-3}
704	2.91×	D_4 Q=19	−0.52%	1.33%	97.7%	2.2×10^{-3}
800	2.56×	TQ $b = 6$	−0.37%	0.96%	98.4%	6.2×10^{-4}
832	**2.46×**	D_4 **Q=38**	+0.41%	0.57%	**99.6%**	$\mathbf{5.4 \times 10^{-4}}$
928	2.21×	TQ $b = 7$	+0.51%	0.51%	100.00%	1.5×10^{-4}
960	2.13×	D_4 Q=76	−0.50%	0.67%	100.00%	1.4×10^{-4}
1056	1.94×	TQ $b = 8$	−0.51%	0.66%	98.83%	3.7×10^{-5}
1088	**1.88×**	D_4 **Q=152**	−0.20%	0.37%	**99.6%**	$\mathbf{3.4 \times 10^{-5}}$

Observations. K-MSE dominates at every near-matched bit tier (7/7; ratio 0.35–0.91×, monotone in bit budget) — the D_4 row sits 3–11% above its paired TQ row in packed bits, so this is near-matched-rate, not iso-rate. The advantage grows with compression aggression (0.91× at 1.88× CR vs. 0.35× at 6.4× CR; the lattice prevents per-coordinate tail truncation from compounding under narrow bit budgets), and at target rel-MSE $\approx 5 \times 10^{-4}$ $q_{\text{range}} = 38$ reaches it at 832 bits vs. an interpolated ∼928 bits for the scalar baseline (a ∼10% bit saving at equal downstream quality).

5.2 Multi-model head-to-head

Table 3 gives the 12 head-to-head ratios across four models at the three canonical operating points.

Table 3: Twelve head-to-head pairs (D_4 variant vs. TurboQuant $k8v4$) at three canonical operating points. Protocol: *snapshot-mode* $n = 4$ passages, boundary $k = 2$ (§4). K-MSE ratio <1 means KakeyaLattice beats the scalar-quantiser baseline on reconstruction; |Δppl| ratio <1 means KakeyaLattice is closer to bf16. "top-1 Δpp" is KakeyaLattice top-1 minus baseline top-1. Bit-point matching is near-matched rather than strictly iso-rate: the KakeyaLattice rows pay 3–11% more bits at the same q-tier (§8). Data: `reports/v1_3_ppl/snapshot_mode_qwen3/multimodel/`.

Model	TQ b	D_4 Q	CR	**K-MSE ratio**	**|Δppl| ratio**	**top-1 Δpp**
Qwen3-4B	4	10	3.56×	**0.639**	**0.385**	−0.78
	6	38	2.46×	**0.868**	**0.308**	+0.78
	8	152	1.88×	**0.911**	**0.912**	+0.39
DeepSeek-R1-Distill-1.5B	4	10	3.56×	**0.639**	**0.187**	+3.91
	6	38	2.46×	**0.868**	2.586	−0.78
	8	152	1.88×	**0.911**	3.080	+0.39
Gemma-4-E4B (hd =256)	4	10	3.66×	**0.638**	**0.376**	−0.39
	6	38	2.51×	**0.866**	**0.819**	+0.00
	8	152	1.91×	**0.909**	1.264	−0.39
GLM-4-9B-Chat	4	10	3.56×	**0.639**	**0.551**	+0.78
	6	38	2.46×	**0.868**	**0.637**	−0.39
	8	152	1.88×	**0.911**	**0.490**	−0.39

Metric	KakeyaLattice wins	tie	baseline wins
K-MSE ratio (< 1.0)	**12/12**	0	0
|Δppl| ratio (< 1.0)	**9/12**	0	3
top-1 pair ($\geq$ baseline)	6/12	2	4

Consolidated verdict. K-MSE dominance is universal (12/12, 10–36% better), mirroring $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ with the expected drift to 0.64 at aggressive $q_{\text{range}} = 10$. $|\Delta\text{ppl}|$ dominance is clean at the deployment operating point ($q_{\text{range}} = 10$, all four models, 1.8–5.3× better). The three near-lossless losses ($q_{\text{range}} = 152$ on DeepSeek-1.5B and Gemma-4-E4B) are below 1% absolute on both sides and therefore inside the FlashAttention bf16 reproducibility floor at $n = 4$; the K-MSE ratio still confirms higher K-fidelity to bf16.

5.3 Theory–experiment reconciliation

Table 4: Theory-to-experiment match for the D_4 K-MSE shaping-gain ratio. Predicted from Proposition 2.2; measured under the snapshot protocol (§4) in three independent environments. The bit-cost row reports both the exact (Eq. (2)) and the slot-packed (integer-aligned, deployment) values; the packed value is what Table 2 reports, per the bit-accounting note in §8.

Quantity	Predicted	Measured
K-MSE ratio at $q_{\text{range}} = 152$ (Qwen3-4B)	0.919	**0.913** (offline 100k samples)
		0.911 (live vLLM, 4 passages)
		0.911 (multi-model mean, Tab. 3)
Shaping gain (dB)	+0.367 dB	+0.381 dB (Qwen3-4B offline)
$\cos(K, \hat{K})$	$\to 1.0$	1.0000
Bit cost (exact, Eq. (2))	1056.3	1056.3
Bit cost (slot-packed, deployment)	—	1088 (paired against TQ $b = 8$ at 1056)

All three measurements agree with the theoretical prediction to within $\sim 1\%$, directly validating Proposition 2.2 at deployment head dimensions.

5.4 128k KV-cache storage

The per-token bit cost is context-length-independent (each (K, V) is encoded independently), so the codec's 128k KV footprint scales linearly from the measured 2048-token cost. Table 5 reports the three canonical operating points with K *and* V both compressed (boundary layers bf16).

Table 5: 128k KV cache storage: baseline bf16 vs. KakeyaLattice (D_4 variant, both K and V compressed, boundary layers $\{0, 1, L-2, L-1\}$ kept bf16). Per-user GiB at ctx = 128,000. Protocol: *snapshot-mode* extrapolation from the measured 2048-token packed-bit cost (§4). Source: `reports/v1_4_release/kv_128k_report/V14_KV_128K_REPORT.md`.

	bf16 128k KV		KakeyaLattice 128k KV			
	GiB	ratio	Q=10	Q=38	Q=152	$\lvert\Delta\text{ppl}\rvert$ best row
Qwen3-4B	18.0	1.0×	2.77×	2.12×	1.71×	0.48% @ Q=152
DeepSeek-R1-Distill-1.5B	3.5	1.0×	2.60×	2.04×	1.67×	0.28% @ Q=152
Gemma-4-E4B (hd =256)	6.0	1.0×	2.53×	2.01×	1.66×	0.15% @ Q=38
GLM-4-9B-Chat	5.0	1.0×	2.83×	2.15×	1.73×	0.90% @ Q=152

Quality summary. Balanced Q=38 is deployment-recommended: $|\Delta\text{ppl}| < 1\%$ on three of four models (GLM-4-9B-Chat: 2.51%), top-1 $\geq 95\%$ universally. Near-lossless Q=152 is universally $< 1\%$ Δppl and $\geq 98\%$ top-1. Aggressive Q=10 is safe on Gemma-4-E4B (0.33%), noticeable on Qwen3-4B/DeepSeek (1.45/2.22%), past-edge on GLM-4-9B-Chat (6.5%).

5.5 Codec wall-clock

Table 6: Encode time ratio D_4 nested lattice variant / TQ $k8v4$, per 2112-token passage on H200. Values < 1.0 mean KakeyaLattice is faster.

Operating point	Qwen3-4B	DeepSeek-1.5B	Gemma-4-E4B	GLM-4-9B	Mean
Q=10 (aggressive)	0.80	0.77	1.18	0.81	**0.89**
Q=38 (balanced)	1.53	1.62	1.45	1.69	1.57
Q=152 (near-lossless)	1.48	1.63	1.47	1.58	1.54

Aggressive point: $\sim 10\%$ faster than the scalar baseline on 3/4 models. Balanced/near-lossless: $\sim 55\%$ slower (cost of the D_4 closest-point search + per-vector qmax cycle). Absolute codec wall time is 0.02–0.04 s per 2112-token passage, negligible against the $\sim$10 s of bf16 prefill.

6 Measured performance of the E_8 nested lattice variant

The codec is parametric in $\Lambda \in \{D_4, E_8\}$; the E_8 variant differs only in the closest-point subroutine. Every result in this section uses the in-forward rigorous protocol (§4; codec applied inside every attention step, no-boundary unless stated, $n = 32$, 95% CI) — deliberately harsher than §5's snapshot protocol, because the E_8 upgrade's practical benefit manifests only under cross-layer error accumulation. E_8's Voronoi cell is closer to a sphere than D_4's, giving +0.29 dB theoretical shaping gain on top of the five engineering levers.

Two E_8 design facts: (i) Conway–Sloane Alg. 5 computes two D_8 candidates (integer + half-integer cosets) per block, at $\approx 2.5\times$ the per-block cost of D_4's single parity-flip; E_8 operates in 8-D blocks so the block count halves, giving a net 1.56–1.80× encode-decode latency ratio (Table 10). (ii) E_8's two-coset union recovers full $\mathbb{Z}^8$ density, losing D_4's -1-bit parity saving. Exact-rate cost is $+D/4 = +32$ bits/vector at every q_{range}; packed (slot-aligned) cost is +32 at low q_{range} narrowing to +16 at high q_{range} as the parity saving is absorbed into ceiling arithmetic (Table 7; CR cost -5–7% at iso-q_{range}).

Table 7: Canonical operating points: D_4 nested lattice variant vs. E_8 nested lattice variant at $D = 128$. bits/tok/head is the *packed* (integer-aligned, deployment) cost including 32 bits fp16 overhead ($\|x\| + q_{\max}$); the exact information-theoretic rate is within one bit per block of the packed value (Eqs. (2), (3)). The packed $E_8 - D_4$ gap is *not* a fixed +32: at low q_{range} it is +32 (parity saving still visible above the rounding floor), at high q_{range} it is +16 (parity saving absorbed into integer alignment). CR is vs. bf16.

q_{range}	D_4 bits	D_4 CR	E_8 bits	E_8 CR	Δ bits
4	416	4.92×	448	4.57×	+32
10	576	3.56×	608	3.37×	+32
38	832	2.46×	848	2.41×	+16
152	1088	1.88×	1104	1.86×	+16

(The packed differential drops from +32 to +16 at large q_{range} because the per-block rates round up to integer bits: $\lceil 4\log_2(2q_{\text{range}}+1)-1\rceil$ for D_4 and $\lceil 8\log_2(2q_{\text{range}}+1)\rceil$ for E_8. At $q_{\text{range}} = 4$ the fractional parts align so that D_4 saves one block bit above E_8, giving +32 across the 16 pair-aligned blocks; at $q_{\text{range}} \geq 38$ the fractional parts align so that the ceilings coincide in one of every pair of blocks, halving the packed gap to +16. The *exact* information-theoretic gap is the constant $+D/4 = +32$ bits independent of q_{range}, matching the abstract claim; the table reports packed values because that is what a deployment KV slot actually consumes.)

6.1 E_8 measured shaping gain

We measure the E_8 nested lattice variant against the D_4 nested lattice variant in the harshest-for-the-codec regime: live vLLM in-forward (codec applied *inside* every attention step, cross-layer error allowed to compound), no-boundary protection (all L layers codec-compressed), $n = 32$ passages with 95% CI (Student-t). Protocol matches the rigorous-evaluation report in `reports/v1_4_release/rigorous_eval/RIGOROUS_EVAL_REPORT.md` §7 on the four-model set.

Table 8: E_8 vs D_4 nested lattice variant per-layer K rel-MSE at matched Q, measured at layer 0 before cross-layer accumulation. Averaged across four models. Theoretical E_8/D_4 ratio is 0.936 (+0.29 dB); the empirical ratio is ≈ 0.65 (+1.87 dB at $Q = 10$), a 6× over-delivery. The excess is attributed to E_8's two-coset candidate pair handling coarse-quant outliers better than D_4's single-flip parity correction.

	$Q = 4$ (aggressive)		$Q = 10$ (moderate)		mean gain	
Model	D_4 K-MSE	E_8 K-MSE	D_4 K-MSE	E_8 K-MSE	ratio	dB
Qwen3-4B	1.25×10^{-2}	8.02×10^{-3}	1.94×10^{-3}	1.27×10^{-3}	0.65	+1.87
DeepSeek-1.5B	2.63×10^{-2}	1.94×10^{-2}	5.36×10^{-3}	3.40×10^{-3}	0.69	+1.65
Gemma-4-E4B	6.73×10^{-2}	4.46×10^{-2}	1.08×10^{-2}	7.16×10^{-3}	0.66	+1.79
GLM-4-9B-Chat	5.13×10^{-2}	3.41×10^{-2}	8.23×10^{-3}	5.38×10^{-3}	0.66	+1.81
Four-model mean	0.66× (+1.81 dB)		0.65× (+1.87 dB)			

Table 9: E_8 vs D_4 nested lattice variant $|\Delta\text{ppl}|$ at matched Q (in-forward, no-boundary, $n = 32$, mean $\pm 95\%$ CI via Student-t). DeepSeek-R1-Distill-1.5B in-forward no-boundary is structurally undeployable for every codec; numbers listed for completeness, not for deployment.

	$Q = 10$			$Q = 4$		
Model	D_4	E_8	reduction	D_4	E_8	reduction
Qwen3-4B	$5.62\% \pm 1.78\%$	$\mathbf{3.85\% \pm 1.25\%}$	$\mathbf{-31.5\%}$	$36.5\% \pm 13.8\%$	$\mathbf{17.0\% \pm 5.3\%}$	$\mathbf{-53.4\%}$
Gemma-4-E4B	$2.22\% \pm 0.60\%$	$\mathbf{1.56\% \pm 0.49\%}$	$\mathbf{-29.7\%}$	$11.2\% \pm 2.5\%$	$\mathbf{5.79\% \pm 1.43\%}$	$\mathbf{-48.3\%}$
GLM-4-9B-Chat	$9.94\% \pm 2.74\%$	$\mathbf{6.96\% \pm 2.03\%}$	$\mathbf{-30.0\%}$	$44.8\% \pm 14.3\%$	$\mathbf{32.4\% \pm 9.4\%}$	$\mathbf{-27.8\%}$
Three-model mean			-30.4%			-43.2%

Super-linear cross-layer amplification. At $Q = 10$ the per-layer MSE ratio 0.65 matches the empirical $|\Delta\text{ppl}|$ ratio 0.70 (essentially linear). At $Q = 4$ the per-layer MSE ratio is still 0.66 but empirical $|\Delta\text{ppl}|$ ratio is 0.47 (-53%) — well below what linear propagation would predict. Cross-layer error compounding is super-linear at aggressive bit budgets, giving the E_8 variant a super-linear advantage in in-forward deployment.

6.2 E_8 latency

Table 10: Pure codec wall time on H200 at $N = 2048$ tokens, $H = 8$ heads, $D = 128$ fp32. Mean and p99 over 500 iterations with `cuda.synchronize()` gates. Bf16 memcpy = 21.5 μs baseline (raw cast to bf16 and back).

Codec	Q/b	bits/head/tok	mean μs	p99 μs	ratio vs the D_4 nested lattice variant at same Q
D_4	4	416	354	2733	1.00×
E_8	4	448	551	808	**1.56×**
D_4	10	576	331	476	1.00×
E_8	10	608	596	988	**1.80×**
D_4	38	832	342	463	1.00×
E_8	38	848	613	1170	1.79×
D_4	152	1088	348	723	1.00×
E_8	152	1104	588	1168	1.69×
TQ scalar	2–8	288–1056	140–144	≤ 210	0.42× (vs the D_4 nested lattice variant)

The 1.56–1.80× ratio is consistent across operating points (p50 stable at 540–560 μs for E_8). Absolute wall time stays below 0.6 ms per 2048-token slice — $< 5\%$ of a typical 10–30 ms H200 vLLM decode step, so not a deployment blocker.

6.3 E_8 long-context retrieval (NIAH)

Long-context retrieval via Kamradt-style NIAH at three context lengths (4k/8k/16k) and three depths (10/50/90%), $n_{\text{trials}} = 3$ per cell, exact-substring match (TurboQuant $b = 2$ requires $k = 2$ even to boot).

Table 11: NIAH overall accuracy across 27 cells per codec per model (ctx $\in \{4k, 8k, 16k\}$ × depth $\in \{0.1, 0.5, 0.9\}$ × $n_{\text{trials}} = 3$). bf16 column is the model's base retrieval ability; codec columns should match bf16 if retrieval is preserved.

Model	bf16	D_4 Q=10	E_8 Q=10	D_4 Q=4	E_8 Q=4	TQ $b = 3$
Qwen3-4B	100%	**100%**	**100%**	**100%**	**100%**	100%
Gemma-4-E4B	100%	**100%**	**100%**	**100%**	**100%**	100%
GLM-4-9B-Chat	100%	**100%**	89%	55.6%	51.9%	55.6%
DeepSeek-1.5B	33.3%*	33.3%	22.2%	11.1%	22.2%	11.1%

* DeepSeek-R1-Distill-Qwen-1.5B's base retrieval ceiling is 33% at our ctx lengths (only depth 0.9 is reachable by the base model); codec numbers are interpretable only relative to that ceiling.

Retrieval is preserved on Qwen3-4B and Gemma-4-E4B at every E_8 operating point. GLM-4-9B-Chat shows a mild E_8 regression at $Q = 10$ (89% vs 100%, one cell failed: ctx=16k, depth=0.9); attributable to GLM-4's partial-rotary RoPE (factor 0.5) interacting with the E_8 Voronoi pattern — the rotated half of the head dimension concentrates the reconstruction error. Use E_8 Q=152 (100% NIAH on all cells) on GLM-4

until a RoPE-aware E_8 ordering is investigated. TurboQuant $b = 2$ is structurally unusable for long-context retrieval on all four models (0/27 cells pass).

7 Caveats and known limits

1. **Snapshot ($n = 4$) vs. in-forward rigorous ($n = 32$).** The snapshot D_4 tables in §5 use $n = 4$ passages; K-MSE is sample-size-independent but $|\Delta\text{ppl}|$ below $\sim 1\%$ absolute is inside the FlashAttention bf16 reproducibility floor and should be read as a tie. The in-forward rigorous tables in §6 use $n = 32$ and 95% CI to eliminate this confound for the E_8-vs-D_4 comparison. The two protocols answer different questions (intrinsic vs. deployment-realistic error); PPL in §5 is a lower bound on in-forward deployment PPL; K-MSE and shaping-gain ratios are not affected by protocol choice.
2. **Head dimension divisibility.** The D_4 codec raises `ValueError` on $d_h \not\equiv 0 \pmod 4$; the E_8 codec on $d_h \not\equiv 0 \pmod 8$. Models with $d_h \in \{96, 176\}$ need a different block structure.
3. **KV-sharing layers.** Gemma-4-E4B's `kv_sharing_target_layer_name` layers are excluded from capture (they re-use another layer's KV cache), matching the model's native semantics.
4. **Kakeya lower bound is conditional at $D \geq 4$.** Proposition 2.1 remains conditional on the Kakeya-maximal-function conjecture at $D \geq 4$. At $D = 3$ the lower bound is unconditional via Wang–Zahl [21]; at deployment head dimensions it is a conditional framing, not a proof of Pareto-optimality. The D_4 *shaping bound* in Proposition 2.2 is unconditional, but it applies to the 4-dimensional block structure of the codec rather than to the full head dimension.

8 Comparison with TurboQuant

TurboQuant [34, 35] is the strongest production-deployed KV codec; its $k8v4$ preset rotates by Sylvester–Hadamard, unit-norm factorises, then quantises K at FP8 E4M3 scalar + V at 4-bit uniform, giving 1056 bits/token/head at $D = 128$ ($8 \cdot 128 + 2 \cdot \text{fp16}$). TurboQuant and KakeyaLattice share four levers (unit-norm factorisation, Hadamard rotation, per-vector $q_{\max}$, joint scale) and differ only in the per-block quantiser: TurboQuant uses int-per-coord $\mathbb{Z}^4$ cells ($G = 1/12$, 0 dB reference), KakeyaLattice uses D_4 closest-point ($G \approx 0.0766$, +0.367 dB) — a drop-in change.

Near-matched rate: the 3–11% premium. At each q-tier the D_4 variant's packed rate sits 3–11% above its paired TurboQuant-b rate because $4\log_2(2q_{\text{range}} + 1) - 1$ rounds up to the next integer (TurboQuant's per-coord uniform bits are integer by construction). At $q_{\text{range}} = 152$ the D_4 exact rate is 1056.3 bits per $D = 128$ vector but the packed rate is 1088, paired against TQ $k8v4$ at exactly 1056. The three canonical operating points sit at packed-rate premia of +11% ($q_{\text{range}} = 10$, 576 vs. ~ 520 interpolated TQ), +4% ($q_{\text{range}} = 38$, 832 vs. 800 at TQ $b = 6$), and +3% ($q_{\text{range}} = 152$, 1088 vs. 1056 at TQ $b = 8$). The Pareto comparisons in Tables 2, 3, 4 are therefore near-matched rate, not iso-rate: at a 3–11% packed-rate premium, KakeyaLattice delivers 8–36% better K-MSE and a strictly better Pareto curve.

Proposed upstream PR. A Triton kernel for Conway–Sloane D_4 closest-point ($\sim$10 lines element-wise); slot-format alignment (32 blocks $\times$33 bits + 2 fp16 scalars, bit-compatible with $k8v4$); new `--kv-cache-dtype` preset `turboquant_d4v4` alongside `turboquant_k8v4`. The change is additive. End-to-end vLLM throughput measurement requires paged-attention-native kernel integration and is out of scope for this paper.

9 Related work

Kakeya problem and harmonic analysis. The Kakeya problem was posed in 1917 [1]; Besicovitch gave the first non-trivial constructions [2, 3]; Fefferman and Córdoba connected it to oscillatory integrals [4, 5]; Bourgain, Wolff, Katz–Łaba–Tao, and Tao advanced high-D and $\mathbb{R}^3$ bounds [6, 7, 8, 9, 10]; Dvir [11, 12] resolved the finite-field analogue via the polynomial method; Bennett–Carbery–Tao, Guth, and Guth–Katz developed the multilinear Kakeya / polynomial-partitioning chain [13, 14, 15]; Hickman–Rogers–Zhang and Zahl pushed high-D bounds [18, 19, 20]; Wang–Zahl [21] resolved the three-dimensional Euclidean case in 2025. The paper relies on three specific links: Guth's multilinear Kakeya endpoint [14], Brascamp–Lieb [16, 17], and Tropp's matrix-concentration bounds [22].

Lattice source coding. Zamir–Feder [24] established the rate–distortion framework; Conway–Sloane [25, 26] gave the D_4 second-moment optimality and the closest-point algorithm. Eyuboglu–Forney [27], Forney [28], and Erez–Zamir [29] developed lattice source/channel coding; QuIP# [30] applied lattice codebooks to LLM weight quantisation. The Halko–Martinsson–Tropp randomised SVD [23] closes the Brascamp–Lieb to matrix-concentration arc numerically and grounded our prior work [33].

KV-cache compression. Token eviction (H_2O [42], Scissorhands [43], StreamingLLM [44]) reduces T, orthogonal to a codec. Per-channel quantisation (KIVI [40]) sits outside the Hadamard-rotation stack. Hadamard-based codecs include *Cache Me If You Must* [41] and TurboQuant [34, 35, 36, 37, 38, 39], our direct comparison baseline (§8). Trained low-rank methods (DMC [45], LESS [46]) are orthogonal to post-hoc codecs. Our prior work [33] used an RSVD + Lloyd-Max codec with a Kakeya-conditional rank bound; the present codec replaces that conjectural bound with the unconditional D_4 shaping bound of Prop. 2.2.

Infrastructure. vLLM [48] is the reference serving stack; transformers [49] supplies weights and tokenisers; FlashAttention [50] is the default attention kernel.

10 Conclusion

KakeyaLattice is a Zamir–Feder nested-lattice KV-cache codec with two lattice variants. The D_4 variant's shaping gain is unconditionally bounded by $G(\mathbb{Z}^4)/G(D_4) \approx 0.919$ via Prop. 2.2; placing it at the shallow end of the multilinear Kakeya $\to$ Brascamp–Lieb $\to$ matrix concentration $\to$ lattice second-moment chain (§2) [14, 16, 22, 26, 21] grounds the design in published theorems without a conditional Kakeya dependency at deployment dimension. Under snapshot evaluation the theoretical 0.919 ratio is recovered within $\sim 1\%$ on every tested model in three independent environments; the codec wins 12/12 on K-MSE and 9/12 on $|\Delta\text{ppl}|$ across four transformer families, and shrinks the 128k KV cache by 2.5–2.8× balanced and 1.66–1.73× near-lossless with both K and V compressed. The E_8 variant extends this with +1.3 to +2.0 dB further per-layer K-MSE gain and a 28–53% in-forward $|\Delta\text{ppl}|$ reduction (four models, $n = 32$, 95% CI); the excess over the +0.29 dB theoretical minimum is super-linear amplification of per-layer error under cross-layer accumulation. Cost is +16–32 packed bits/vector and 1.56–1.80× encode latency; long-context retrieval is preserved on three of four models. We frame the codec as an additive near-matched-rate drop-in PR over the production-deployed TurboQuant kernel (§8).

Reproducibility and distortion choice. All code, unit tests, per-model JSON, four per-architecture attention hooks, and the multi-model / NIAH / latency harnesses are released under Apache-2.0 in `kakeyalattice` (D_4: `V14KakeyaZamirLatticeGPU`; E_8: `V15KakeyaZamirE8GPU`); full manifest §A. We report MSE rather than inner-product distortion because $|q^\top(k - \hat{k})| \le \left\|k - \hat{k}\right\|$ for unit-norm q, so MSE is a conservative upper bound on attention-logit perturbation; measured $|\Delta\text{ppl}|$ and top-1 agreement confirm this proxy is tight in practice.

References

[1] Kakeya, S. Some problems on maxima and minima regarding ovals. *Tohoku Science Reports*, 6:71–88, 1917.

[2] Besicovitch, A. S. On Kakeya's problem and a similar one. *Mathematische Zeitschrift*, 27:312–320, 1928 (original work 1919).

[3] Besicovitch, A. S. On the fundamental geometrical properties of linearly measurable plane sets of points. *Mathematische Annalen*, 98:422–464, 1928.

[4] Fefferman, C. The multiplier problem for the ball. *Annals of Mathematics*, 94(2):330–336, 1971.

[5] Córdoba, A. The Kakeya maximal function and the spherical summation multipliers. *American Journal of Mathematics*, 99(1):1–22, 1977.

[6] Bourgain, J. Besicovitch type maximal operators and applications to Fourier analysis. *Geometric and Functional Analysis*, 1(2):147–187, 1991.

[7] Wolff, T. An improved bound for Kakeya type maximal functions. *Revista Matemática Iberoamericana*, 11(3):651–674, 1995.

[8] Katz, N. H., Łaba, I., Tao, T. An improved bound on the Minkowski dimension of Besicovitch sets in $\mathbb{R}^3$. *Annals of Mathematics*, 152(2):383–446, 2000.

[9] Katz, N. H., Tao, T. Some connections between Falconer's distance set conjecture and sets of Furstenburg type. *New York Journal of Mathematics*, 7:149–187, 2001.

[10] Tao, T. From rotating needles to stability of waves: emerging connections between combinatorics, analysis, and PDE. *Notices of the AMS*, 48(3):294–303, 2001.

[11] Dvir, Z. On the size of Kakeya sets in finite fields. *Journal of the American Mathematical Society*, 22(4):1093–1097, 2009.

[12] Dvir, Z., Kopparty, S., Saraf, S., Sudan, M. Extensions to the method of multiplicities, with applications to Kakeya sets and mergers. *SIAM Journal on Computing*, 42(6):2305–2328, 2013.

[13] Bennett, J., Carbery, A., Tao, T. On the multilinear restriction and Kakeya conjectures. *Acta Mathematica*, 196(2):261–302, 2006.

[14] Guth, L. The endpoint case of the Bennett–Carbery–Tao multilinear Kakeya conjecture. *Acta Mathematica*, 205(2):263–286, 2010.

[15] Guth, L., Katz, N. H. On the Erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.

[16] Bennett, J., Carbery, A., Christ, M., Tao, T. The Brascamp–Lieb inequalities: finiteness, structure and extremals. *Geometric and Functional Analysis*, 17(5):1343–1415, 2008.

[17] Carbery, A., Valdimarsson, S. I. The endpoint multilinear Kakeya theorem via the Borsuk–Ulam theorem. *Journal of Functional Analysis*, 264(7):1643–1663, 2013.

[18] Hickman, J., Rogers, K. M., Zhang, R. Improved bounds for the Kakeya maximal conjecture in higher dimensions. *American Journal of Mathematics*, 144(6):1511–1560, 2022.

[19] Zahl, J. On the Wolff circular maximal function. *Illinois Journal of Mathematics*, 56(4):1281–1295, 2012.

[20] Zahl, J. A new bound on Kakeya sets in $\mathbb{R}^3$. *arXiv preprint arXiv:2104.13528*, 2022.

[21] Wang, H., Zahl, J. Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions. *arXiv preprint arXiv:2502.17655*, 2025.

[22] Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012.

[23] Halko, N., Martinsson, P.-G., Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[24] Zamir, R., Feder, M. On lattice quantization noise. *IEEE Transactions on Information Theory*, 42(4):1152–1159, 1996.

[25] Conway, J. H., Sloane, N. J. A. Fast quantizing and decoding algorithms for lattice quantizers and codes. *IEEE Transactions on Information Theory*, 28(2):227–232, 1982.

[26] Conway, J. H., Sloane, N. J. A. *Sphere Packings, Lattices and Groups*, 3rd ed. Springer, 1999.

[27] Eyuboglu, M. V., Forney, G. D. Lattice and trellis quantization with lattice- and trellis-bounded codebooks—high-rate theory for memoryless sources. *IEEE Transactions on Information Theory*, 39(1):46–59, 1993.

[28] Forney, G. D. Coset codes—part I: introduction and geometrical classification. *IEEE Transactions on Information Theory*, 34(5):1123–1151, 1988.

[29] Erez, U., Zamir, R. Achieving $1/2\log(1+\mathrm{SNR})$ on the AWGN channel with lattice encoding and decoding. *IEEE Transactions on Information Theory*, 50(10):2293–2314, 2004.

[30] Chee, J., Cai, Y., Kuleshov, V., De Sa, C. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. *ICML*, 2024.

[31] Tseng, A., Chee, J., Sun, Q., Kuleshov, V., De Sa, C. QuIP: 2-bit quantization of large language models with guarantees. *NeurIPS*, 2023.

[32] Li, A. KakeyaLattice: the canonical implementation. Open-source release, *LLM-KV–Cache-compress*, April 2026. Python package `kakeyalattice` (classes `V14KakeyaZamirLatticeGPU` and `V15KakeyaZamirE8GPU`); multi-model measurement harness `benchmarks/rigorous_eval.py`; four per-architecture attention-module patches in `vllm_backend/kakeya_v1_4_snapshot/snapshot_hook.py`. Release tags: `v1.4` (first D_4 release, commit `6b02711`) and `v1.5` (first E_8 release).

[33] Li, A. Randomized Kakeya Skeletons for LLM KV Cache Compression: Algorithm and Rate–Distortion Boundary. Prior unpublished draft, superseded by this paper.

[34] vibhavagarwal5 *et al.* TurboQuant: 2-bit KV cache compression with 4x capacity. *vLLM merged attention backend*, 2026. (Merged into vllm-project/vllm main on 2026-04-15.)

[35] vibhavagarwal5. [Attention Backend] TurboQuant: 2-bit KV cache compression with 4x capacity. *vllm-project/vllm Pull Request #38479*, 2026. https://github.com/vllm-project/vllm/pull/38479.

[36] Alistarh, D. [Attention] TurboQuant: remove redundant random signs, add prior art attribution. *vllm-project/vllm Pull Request #40194*, 2026.

[37] aditi-amd. [ROCm] Fix TurboQuant on ROCm: backend routing, flash-attn compat. *vllm-project/vllm Pull Request #39953*, 2026.

[38] mgoin. Fix TURBOQUANT backend selection in `cuda.py`. *vllm-project/vllm Pull Request #40060*, 2026.

[39] huangzhilin-hzl. [TurboQuant] Add NIAH long-context eval for TurboQuant KV cache. *vllm-project/vllm Pull Request #40122*, 2026.

[40] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.

[41] Shutova, E. *et al.* Cache Me If You Must: Adaptive KV Cache Quantization for LLMs. *arXiv preprint arXiv:2501.19392*, 2025.

[42] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

[43] Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *NeurIPS*, 2023.

[44] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[45] Nawrot, P., Lancucki, A., Chochowski, M., Tarjan, D., Ponti, E. M. Dynamic memory compression: Retrofitting LLMs for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.

[46] Dong, H., Yang, X., Zhang, Z., Wang, Z., Chi, Y., Chen, B. Get more with LESS: Synthesizing recurrence with KV cache compression for efficient LLM inference. *ICML*, 2024.

[47] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.

[48] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., Stoica, I. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[49] Wolf, T. *et al.* Transformers: State-of-the-art natural language processing. *EMNLP (Demo)*, 2020.

[50] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

A Reproducibility manifest

The full multi-model benchmark for both lattice variants uses the in-forward rigorous evaluation harness (§4). The snapshot-protocol D_4 tables in §5 are reproducible by the same harness with `--mode snapshot --boundary-size 2 --n-passages 4`:

```
cd LLM-KV--Cache-compress
pip install -e kakeyalattice
pip install -e vllm_backend
export VLLM_ENABLE_V1_MULTIPROCESSING=0 KAKEYA_SNAPSHOT_QWEN3=1

# In-forward rigorous (n=32, 95% CI): D4, E8, and TurboQuant, same run.
# --q-values selects D4 operating points; --v15-q-values selects E8
# operating points; --tq-b-values selects TurboQuant bit widths.
python benchmarks/rigorous_eval.py \
    --model-path <HF-id> --model-name <short>_nobdry \
    --mode inforward --no-boundary \
    --q-values 4,10 --v15-q-values 4,10 --tq-b-values 3 \
    --kv-modes KV \
    --ctx-len 2048 --n-eval 64 --n-passages 32 \
    --out-dir reports/rigorous_eval

# TurboQuant b=2 guardrail (requires boundary=2 to boot):
python benchmarks/rigorous_eval.py \
    --model-path <HF-id> --model-name <short>_tqb2_bdry2 \
    --mode inforward --boundary-size 2 \
    --q-values "" --v15-q-values "" --tq-b-values 2 \
    --kv-modes KV \
    --ctx-len 2048 --n-eval 64 --n-passages 32 \
    --out-dir reports/rigorous_eval

# Pure codec latency (no model needed):
python benchmarks/e8_latency_benchmark.py --n-iters 500

# NIAH long-context retrieval:
python benchmarks/niah_eval.py \
    --model-path <HF-id> --model-name <short> \
    --mode inforward --boundary-size 2 --n-trials 3 \
    --ctx-lengths 4096,8192,16384 --depths 0.1,0.5,0.9 \
    --q-values 4,10 --v15-q-values 4,10 --tq-b-values 2,3 \
    --out-dir reports/rigorous_eval/niah

# Frozen sha256 parity (bit-level regression gate):
python benchmarks/e8_parity_and_smoke.py
```

Replace `<HF-id>` with one of `Qwen/Qwen3-4B`, `deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B`, `google/gemma-4-E4B`, or `zai-org/GLM-4-9B-Chat` (add `--trust-remote-code` for the last). Raw per-passage JSON, full stdout logs, and frozen codec-output hashes are committed under `reports/` at the release tags listed in Appendix B. The CLI flag names `--v15-q-values` are verbatim repository identifiers of the E_8-variant codec registered at release tag `v1.5`; similarly, directory names `reports/v1_4_release/` and `reports/v1_5_release/` are on-disk paths that match the corresponding release tags.

B Implementation identifiers

The paper is agnostic to version labelling: the two codec variants are the D_4 *nested lattice* and the E_8 *nested lattice*, and every result table cites its protocol. This appendix lists only the repository-level identifiers needed to reproduce the bit-identical codec output.

- Project name: KakeyaLattice.
- Python package: `kakeyalattice`.

- D_4 variant: class `V14KakeyaZamirLatticeGPU`, module `kakeyalattice.v1_4_kakeya_zamir_lattice_gpu`, release tag `v1.4` (commit `6b02711`).
- E_8 variant: class `V15KakeyaZamirE8GPU`, module `kakeyalattice.v1_5_kakeya_zamir_e8_gpu`, release tag `v1.5`.

The `v1.4` / `v1.5` strings appear only as git release tags and as the `V14`/`V15` class-name prefixes chosen by the repository authors; the paper itself references the codec variants exclusively by their lattice names.

C Canonical operating points

For $D = 128$ (head_dim of Qwen3-4B, DeepSeek-R1-Distill-1.5B, and GLM-4-9B-Chat):

	D_4 nested lattice variant		E_8 nested lattice variant		
q_{range}	bits	CR	bits	CR	Role
152	1088	1.88×	1104	1.86×	Near-lossless
76	960	2.13×	976	2.10×	
38	832	2.46×	848	2.41×	D_4 **balanced**
19	704	2.91×	720	2.84×	
10	576	3.56×	608	3.37×	E_8 **balanced**
5	448	4.57×	464	4.41×	
4	416	4.92×	448	4.57×	Aggressive compression
2	320	6.40×	336	6.10×	Extreme compression

For Gemma-4-E4B's heterogeneous head_dim (20 sliding-attention layers with $d_h = 256$, 4 full-attention layers with $d_h = 512$), both codecs emit per-layer bit budgets via the `LatticeCodebook` dispatcher in `kakeyalattice.lattice_codebooks`; the aggregated 128k storage figures in Table 5 use those per-layer values.

Deployment recommendation. Use E_8 **Q=**10 (CR 3.37×, $|\Delta\text{ppl}|$ typically 1.6–6.9%) as the default head-of-line operating point. Use E_8 Q=4 (CR 4.57×) when aggressive compression is non-negotiable and $|\Delta\text{ppl}|$ of 6–32% is acceptable on the target model family. Use E_8 Q=152 (CR 1.86×, $|\Delta\text{ppl}| < 1\%$) when near-lossless is required. Stay on the D_4 nested lattice variant for latency-critical paths where the 1.56–1.80× E_8 encode cost is unacceptable.